Marketplace B2B para anúncios de alimentos naturais, orgânicos, sustentáveis e artesanais.

O projeto possui:

- Backend em Node.js + Express
- Banco de dados SQLite
- Frontend em HTML, CSS e JavaScript puro
- Autenticação com JWT
- Senhas criptografadas com bcrypt
- Upload de imagens com multer
- Controle de acesso por perfil: comprador, fornecedor e administrador

---

1. Requisitos para rodar o projeto

Antes de começar, instale:

- Node.js
- NPM
- Git

Para verificar se o Node e o NPM estão instalados:

```
node -v
npm -v

# 🌿 Kakuab Market - Backend API

Este é o repositório do backend para o projeto **Kakuab Market** – um marketplace
B2B focado na conexão entre fornecedores e compradores de produtos naturais,
orgânicos e sustentáveis.

A API foi construída para rodar em um servidor local e fornece todas as
funcionalidades necessárias para o sistema de usuários, autenticação,
gerenciamento de anúncios, upload de imagens, avaliações e favoritos.

## 🛠️ Tecnologias Utilizadas

- **Node.js**: Ambiente de execução Javascript no servidor.
- **Express.js**: Framework para roteamento e construção da API RESTful.
- **SQLite3**: Banco de dados relacional leve e embutido.
- **JSON Web Token (JWT)**: Autenticação de usuários via tokens de sessão.
- **Bcrypt**: Criptografia segura para armazenamento de senhas.
- **Multer**: Middleware para o recebimento e upload de imagens (arquivos físicos
na pasta `uploads/`).
- **Nodemon**: Reinício automático do servidor durante o desenvolvimento local.

---

## 🗂️ Estrutura do Projeto
```

A arquitetura do projeto segue o padrão estrutural **MVC (Model-View-Controller)** adaptado para uma API (Model, Controller, Routes).

```text
/
├── Frontend_conectado/   # Código do Frontend (HTML, CSS, JS)
├── uploads/              # Armazenamento local das imagens recebidas dos anúncios
├── src/
│   ├── config/           # Configuração de conexão com o banco de dados (database.js)
│   ├── controllers/      # Regras de negócio. Intermedeiam rotas e models.
│   ├── middlewares/      # Interceptadores (validação de JWT, checagem de regras (admin, fornecedor), Multer para imagens)
│   ├── models/           # Lógica do banco de dados (Query SQL para Usuarios, Anuncios, Avaliacoes, etc)
│   ├── routes/           # Definição dos endpoints da API (/api/users, /api/anuncios...)
│   └── server.js         # Ponto de entrada (Entrypoint) onde o servidor Express é instanciado
├── .env                  # Variáveis de ambiente (Segredos, Porta)
└── package.json          # Lista de pacotes e scripts de execução
```

🚀 Como Funciona e Como Rodar Localmente

Sim, o sistema atual roda localmente na sua máquina (`http://localhost:3000`).

1. Pré-requisitos

Ter o Node.js instalado.

2. Instalação

Abra o terminal na pasta raiz do projeto e execute:

```
npm install
```

3. Configuração de Variáveis de Ambiente

Certifique-se de ter o arquivo `.env` na raiz do projeto com o seguinte conteúdo mínimo:

```
PORT=3000
JWT_SECRET=UmaChaveMuitoSeguraParaCriptografia
```

4. Iniciando o Servidor

Para ligar o servidor em modo de desenvolvimento (que reinicia automaticamente quando algum arquivo é salvo):

```
npm run dev
```

Nota: O banco de dados SQLite (`banco_kakuab.sqlite`) é gerado automaticamente na pasta `src/database` na primeira vez que o sistema tenta se conectar com sucesso.

🔐 Sistemas de Autenticação e Perfis

O backend divide os acessos usando o conceito de **Role-Based Access Control (RBAC)** através do `roleMiddleware`:

1. **Admin (`admin`)**: Possui permissão máxima. Consegue visualizar o dashboard administrativo, bloquear/desbloquear usuários e aprovar/reprovar anúncios.
2. **Fornecedor (`fornecedor`)**: Consegue criar anúncios de produtos, editar e deletar seus próprios produtos.
3. **Comprador (`comprador`)**: Consegue favoritar produtos e realizar avaliações/comentários com estrelas nas páginas dos anúncios.

Fluxo de Login (Como o Front se comunica)

1. O Front envia e-mail e senha para `POST /api/auth/login`.
2. O Backend confere o Hash com o Bcrypt e, se bater, gera um Token (JWT).
3. O Token é salvo no `localStorage` do frontend do usuário.
4. O Front envia esse Token no Header de cada requisição futura (`Authorization: Bearer <token>`) para provar que está logado.

📦 Gestão de Anúncios e Uploads

A plataforma armazena e serve as fotos do produto por conta própria.

- Quando um fornecedor cria o anúncio (`POST /api/anuncios`), o `Multer` captura a imagem e a salva dentro da pasta raiz `/uploads`.
- O caminho do arquivo e as informações em texto vão para as tabelas `anuncios` e `anuncio_imagens` do banco de dados SQLite.
- As imagens são expostas de volta para a vitrine através de pastas estáticas (servidas por meio de configuração no `server.js`).